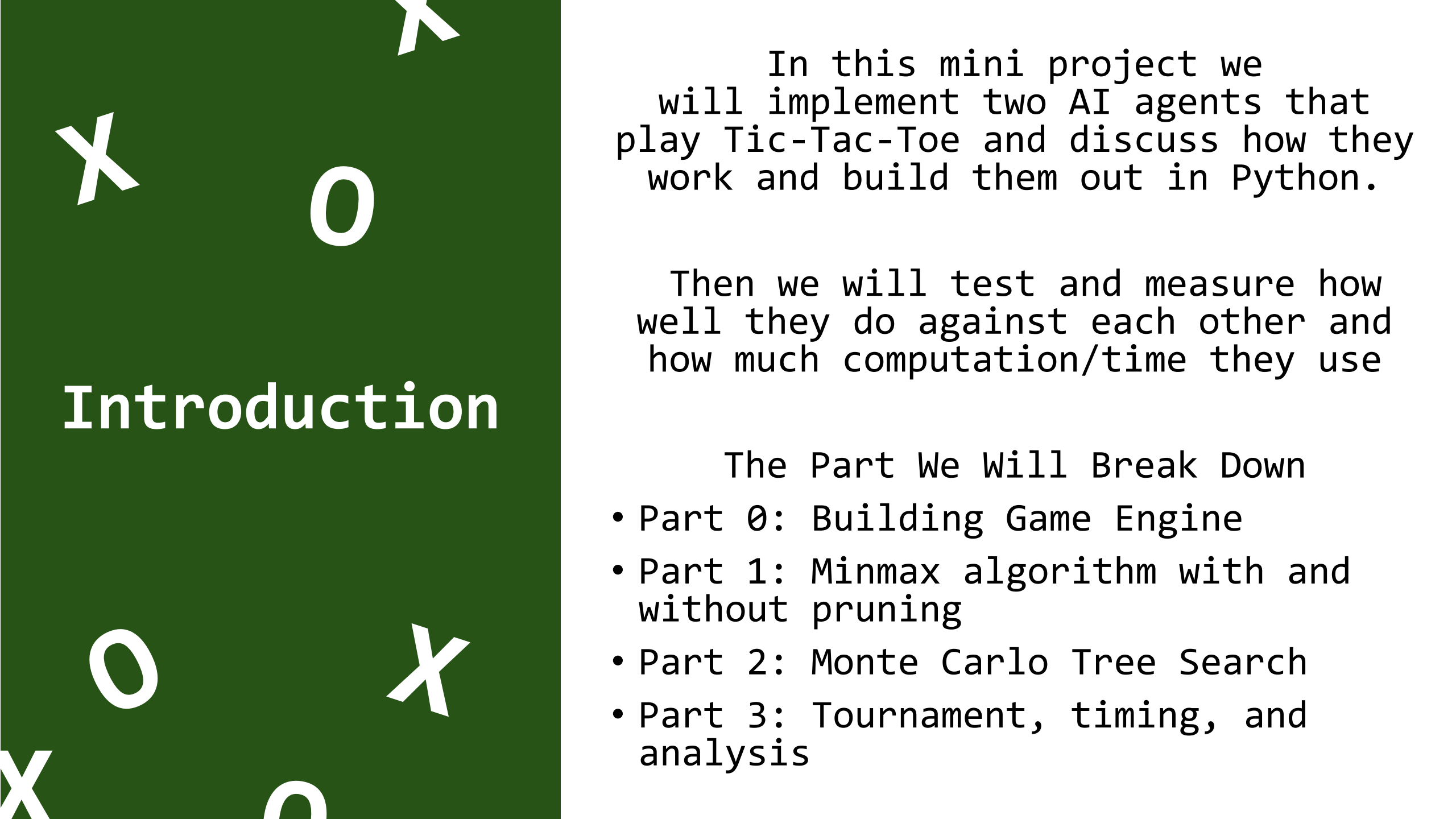


Mini-Project 3

Adversarial Search for Tic-Tac-Toe

Nickolas Diaz		Andres Saldarriaga
Ethan Ashton	Caitlyn Sherman	Tris Replogle



Introduction

In this mini project we will implement two AI agents that play Tic-Tac-Toe and discuss how they work and build them out in Python.

Then we will test and measure how well they do against each other and how much computation/time they use

The Part We Will Break Down

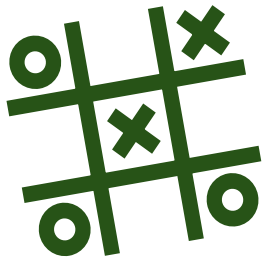
- Part 0: Building Game Engine
- Part 1: Minmax algorithm with and without pruning
- Part 2: Monte Carlo Tree Search
- Part 3: Tournament, timing, and analysis

Introduction

Tic-Tac-Toe, although simple, its potential combinations are many, think there are 9 ternary options for the game board, making the calculation of 3^9
= 19,683 board combinations.

Talking about potential moves is around 9 factorial, equaling 362,880 moves, though the actual number is much smaller since it includes moves that are played after winning.

Since there are so many combinations of moves using a decision tree would be too hard to implement since there are too many options.



Game Engine



```
class TicTacToe:
    def __init__(self):
        self.board: list[int] = [0] * 9 # Board is a list of 9 elements:
        self.board: list[int] = [0] * 9 # 0 = empty, 1 = X, -1 = O

    def get_legal_moves(self) -> list[int]:
    def make_move(self, move: int) -> 'TicTacToe':
    def is_terminal(self) -> bool:
    def check_winner(self) -> int:
    def display(self) -> None:
```

The Mini-Max Algorithm would be a solution to solve Tic-Tac-Toe, it is a decision-making algorithm.

Minimizes the possible loss in a worst-case scenario and maximize the potential gain.

It involves 2 players/agents, if it is the player's turn, pick the max move where the most utility is gained

If it is the opponent's turn, pick the min move where the least utility is gained.

Basic MinMax



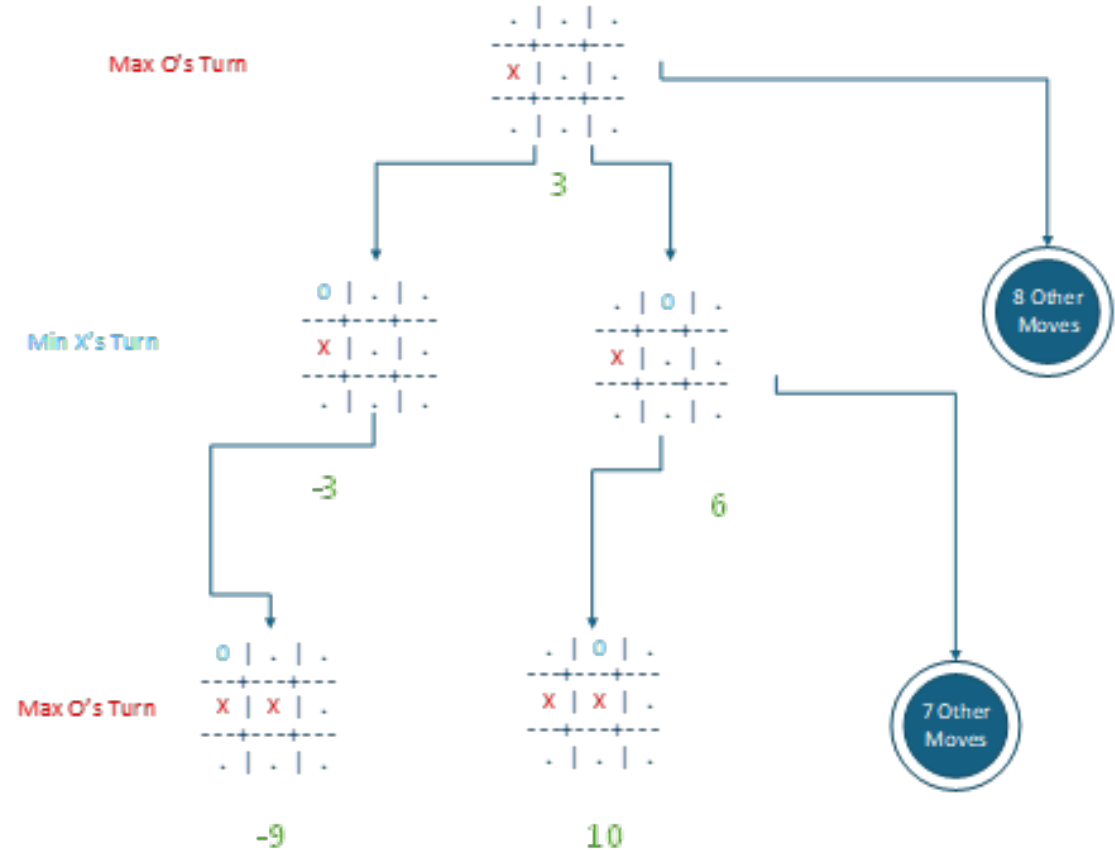
Basic minimax Code

```
def minimax_helper(state: TicTacToe) -> tuple[int, int]:
    best_move = -1
    best_value = -sys.maxsize * state.current_player
    if state.is_terminal():
        return state.utility(), -1

    for move in state.get_legal_moves():
        child = state.make_move(move)
        potential_value, _ = minimax_helper(child)

        if state.current_player == 1:
            if potential_value > best_value:
                best_value = potential_value
                best_move = move
        else:
            if potential_value < best_value:
                best_value = potential_value
                best_move = move

    return best_value, best_move
```



Alpha-beta pruning

Alpha-Beta Pruning: further optimizes the Mini-Max Algorithm, it prunes branches that do not need to be calculated.

It uses two extra inputs, one for alpha which is the best value that the maximizing player can guarantee and beta which is the best value the minimizing player can guarantee.

If one of the player's options is lower than the alpha then it is skipped, also If one of the opponents' options is higher than the alpha then it is skipped.

This pruning greatly improves the computational side of running it since it can get exponentially expensive over time.



Alpha-beta pruning Code

```
def better_alphabeta_helper(state: TicTacToe, alpha: int, beta: int) -> tuple[int, int]:
    if state.is_terminal():
        return state.utility(), -1

    best_move = -1 # -inf for Maximizer, +inf for Minimizer
    best_value = -sys.maxsize * state.current_player

    for move in state.get_legal_moves():
        child = state.make_move(move)
        potential_value, _ = better_alphabeta_helper(child, alpha, beta)

        if state.current_player == 1: # Maximizer (X)
            if potential_value > best_value:
                best_value = potential_value
                best_move = move
            if best_value >= beta:
                return best_value, best_move
            alpha = max(alpha, best_value)
        else: # Minimizer (O)
            if potential_value < best_value:
                best_value = potential_value
                best_move = move
            if best_value <= alpha:
                return best_value, best_move
            beta = min(beta, best_value)

    return best_value, best_move
```

RESULTS:

The total number of nodes evaluated by plain minimax: 549,947

The total number of nodes evaluated by alpha-beta: 18,298

The percentage of nodes pruned: 96.67%

Time complexity of minimax: $O(b^m)$

Time complexity of alphabeta: $O(b^{m/2})$

Monte Carlo Tree Search

Estimates the value of a move by simulating many random games from the current position and seeing the outcomes. The value of a state is the fraction of simulations that resulted in a win.

MCTS was run with 100, 1000, and 10000 iterations. The quality of play went down as the iterations increased somewhere between 1000 and 10000.

Increasing the value of c cause more exploitation with lowering the value leading to more exploration.

The algorithm chooses the most visited spot because the spot with the most visits has more options for winning. This is different from the win rate as it could take an option different from one that would allow it to directly win. For example, it could block a move instead of making a final winning move instead.

Not needing an evaluation function means that it will spend less time double checking the move it has made and that it will explore options that other algorithms would not choose.

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	97	0	3
Random (X) vs Minimax (O)	0	77	23
Minimax (X) vs MCTS (O)	12	0	88
MCTS (X) vs Minimax (O)	0	1	99
Perfect (X) vs Random (O)	98	0	2
Random (X) vs Perfect (O)	0	88	12
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	0	100
Random (X) vs MCTS (O)	5	94	1
MCTS (X) vs Random (O)	100	0	0

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	97	0	3
Random (X) vs Minimax (O)	0	79	21
Minimax (X) vs MCTS (O)	9	0	91
MCTS (X) vs Minimax (O)	0	2	98
Perfect (X) vs Random (O)	98	0	2
Random (X) vs Perfect (O)	0	85	15
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	3	97
Random (X) vs MCTS (O)	4	94	2
MCTS (X) vs Random (O)	98	0	2

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	98	0	2
Random (X) vs Minimax (O)	0	76	24
Minimax (X) vs MCTS (O)	10	0	90
MCTS (X) vs Minimax (O)	0	2	98
Perfect (X) vs Random (O)	100	0	0
Random (X) vs Perfect (O)	0	90	10
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	2	98
Random (X) vs MCTS (O)	4	90	6
MCTS (X) vs Random (O)	99	0	1

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	97	0	3
Random (X) vs Minimax (O)	0	78	22
Minimax (X) vs MCTS (O)	12	0	88
MCTS (X) vs Minimax (O)	0	6	94
Perfect (X) vs Random (O)	93	0	7
Random (X) vs Perfect (O)	0	84	16
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	4	96
Random (X) vs MCTS (O)	3	90	7
MCTS (X) vs Random (O)	99	0	1

Monte Carlo Tree Search Code

```
import math
import random
from typing import Optional
```

```
class MCTSNode:
    def __init__(self, state: TicTacToe, parent: Optional['MCTSNode'] = None, move: Optional[int] = None):
        self.state = state
        self.parent = parent
        self.move = move
        self.children: list['MCTSNode'] = []
        self.wins = 0
        self.visits = 0
        self.untried_moves = state.get_legal_moves()

    def is_fully_expanded(self) -> bool:
        # Return True if every legal move from
        # this state has a corresponding child.

        # Since we pop/remove moves from untried_moves when we create
        # a new child, the node is fully expanded when this list is empty.
        return len(self.untried_moves) == 0

    def best_child(self, c: float = 1.41) -> Optional['MCTSNode']:
        # Return the child with the highest UCB1
        # score. Use the formula from Section 5.4.
        best_score: float = float('-inf')
        best_node: Optional['MCTSNode'] = None

        for child in self.children:
            # If a child somehow has 0 visits, it has infinite exploration value.
            # (In standard MCTS, best_child is usually only called on fully expanded nodes)
            if child.visits == 0:
                return child

            # UCB1 Calculation
            exploit: float = child.wins / child.visits
            explore: float = c * math.sqrt(math.log(self.visits) / child.visits)
            ucb1_score: float = exploit + explore

            if ucb1_score > best_score:
                best_score = ucb1_score
                best_node = child

        return best_node

    def best_move(self) -> Optional[int]:
        # Return the move leading to the child
        # with the most visits (not the highest
        # win rate).
        if not self.children:
            return None

        # max() makes it easy to find the child with the highest visit count
        most_visited_child = max(self.children, key=lambda child: child.visits)
        return most_visited_child.move
```

```
def select(node: MCTSNode) -> Optional[MCTSNode]:
    # Starting from the given node, repeatedly
    # choose the best child (by UCB1) until you
    # reach a node that is not fully expanded
    # or that represents a terminal state.

    while node.is_fully_expanded() and not node.state.is_terminal():
        node = node.best_child()

    return node

def expand(node: MCTSNode) -> MCTSNode:
    # Choose one of the untried moves, create a
    # new child node for it, and return the child.

    # Randomly select an untried move and remove it from the list
    # (This ensures we eventually try all moves and triggers is_fully_expanded)
    move: int = random.choice(node.untried_moves)
    node.untried_moves.remove(move)

    # Generate the new game state that results from this move
    new_state: TicTacToe = node.state.make_move(move)

    # Create the child node.
    # Pass the new state, the current node as the parent, and the move used.
    child_node: MCTSNode = MCTSNode(state=new_state, parent=node, move=move)

    # Add the newly created child to the parent's list of children
    node.children.append(child_node)

def simulate(state: TicTacToe) -> int:
    # From the given state, play a random game to
    # completion. At each step, choose a uniformly
    # random legal move. Return the utility.

    # Keep playing random moves until someone wins or it's a draw
    while not state.is_terminal():
        possible_moves: list[int] = state.get_legal_moves()
        move: int = random.choice(possible_moves)

        # Advance the simulation to the next state
        state: TicTacToe = state.make_move(move)

    # The game is over. Return the result (+1, -1, or 0)
    return state.utility()
```

```
def backpropagate(node: MCTSNode, result: int) -> None:
    # Walk from the given node up to the root.
    # Increment the visit count of every node.
    # Increment the win count only for nodes
    # where the result was favorable for the
    # player who made the move.

    while node is not None:
        node.visits += 1

        # The player who made the move to reach this specific node
        # is the opposite of the player whose turn it is currently.
        player_who_moved: int = -node.state.current_player

        # If the result (+1 for X, -1 for O) matches the player
        # who made the move, it was a winning simulation for them.
        if result == player_who_moved:
            node.wins += 1

        # (Optional but common: You could add an `elif result == 0:` here
        # and add 0.5 to `node.wins` so the AI prefers draws over losses.)

        # Move one step up the tree
        node = node.parent

def mcts(state: MCTSNode, iterations: int = 1000) -> int:
    root = MCTSNode(state)
    for _ in range(iterations):
        # 1. Select
        leaf: MCTSNode = select(root)
        # 2. Expand (if not terminal)
        if not leaf.state.is_terminal():
            leaf = expand(leaf)
        # 3. Simulate
        result: int = simulate(leaf.state)
        # 4. Back-propagate
        backpropagate(leaf, result)
    return root.best_move()
```



Tournament, Timing, and Analysis

The agent that wins against random the most would be the MCTS agent for playing X and Y. Which was expected.

MCTS lost 10 times when O and 2 times when X against MinMax and ties for the rest of the 100 times. We could not find an iteration amount that led to a win for MCTS.

After 2000 iterations, MCTS begins to become slower than the minmax agent.

In games with large branching factor, you would want to use MCTS as its focus on promising further results on trees instead of fully exploring every single option means that it will more efficiently be able to find optimal solutions, especially compared to minmax.

MCTS works better in scenarios where evaluation functions may be too complex to create, when the problem features a large branching factor, or when there's limited time or computational power to use with the model. Minmax tends to be better in scenarios with low branching factors, ones with simple and strong evaluation functions, and ones where high precision is required in moves.

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	98	0	2
Random (X) vs Minimax (O)	0	76	24
Minimax (X) vs MCTS (O)	10	0	90
MCTS (X) vs Minimax (O)	0	2	98
Perfect (X) vs Random (O)	100	0	0
Random (X) vs Perfect (O)	0	90	10
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	2	98
Random (X) vs MCTS (O)	4	90	6
MCTS (X) vs Random (O)	99	0	1



